

Chapitre 8.1: Les concepts de base PowerShell pour Active Directory

Ce chapitre vous introduira aux concepts fondamentaux de PowerShell dans le contexte d'Active Directory. Vous n'avez pas besoin d'être développeur pour comprendre et utiliser ces concepts !

1. ♦ Les variables : des boîtes de rangement

Une variable est simplement un conteneur qui stocke une information. Pensez-y comme à une boîte étiquetée où vous rangez temporairement des données.

Création d'une variable

```
# Créer une variable pour stocker un nom d'utilisateur
$nomUtilisateur = "jean.dupont"

# Afficher le contenu de la variable (plusieurs méthodes)
Write-Host $nomUtilisateur           # Méthode 1: Utiliser Write-Host
(affichage formaté)
$nomUtilisateur                     # Méthode 2: Simplement taper le nom
de la variable
Write-Output $nomUtilisateur         # Méthode 3: Utiliser Write-Output
(pour le pipeline, on le verra plus tard)
echo $nomUtilisateur                # Méthode 4: Utiliser l'alias 'echo'
(comme Write-Output)
```

Remarquez le symbole **\$** qui indique qu'il s'agit d'une variable.

Exercice 1.1 : Créez votre première variable

1. Créez une variable **\$monDomaine** contenant le nom de votre domaine AD
2. Affichez son contenu
3. Modifiez sa valeur et affichez-la à nouveau

Exercice 1.2 : Variables et concaténation

1. Créez une variable **\$prenom** contenant votre prénom
2. Créez une variable **\$nom** contenant votre nom de famille
3. Créez une variable **\$nomComple**t qui combine les deux avec un espace entre eux (indice : utilisez **\$prenom + " " + \$nom**)
4. Créez une variable **\$loginAD** qui serait votre identifiant AD au format **prenom.nom** (comme "jean.dupont")
5. Affichez un message qui dit "Bonjour [nomComple], votre login est [loginAD]"

Variables et commandes AD

Les variables sont particulièrement utiles pour stocker les résultats de vos commandes AD :

```
# Stocker un utilisateur dans une variable avec toutes ses propriétés
$utilisateur = Get-ADUser -Identity "jean.dupont" -Properties *
$groupe = Get-ADGroup -Identity "CN=GG-EU-RH-Admin,OU=Groups,OU=RH,OU=EU,DC=computerelectronics,DC=be" -Properties *
```

Note: ceci était une recherche par **identifiant**.

Le paramètre -Identity : Ce paramètre permet de spécifier quel objet AD vous voulez obtenir/modifier. Il accepte plusieurs formats d'identification. Par exemple... si on a le groupe "GG-EU-IT-Users", on peut le trouver de plusieurs manières en utilisant le paramètre -Identity :

- **Nom :** Simplement le nom du groupe (ex: -Identity "GG-EU-IT-Users")
- **SamAccountName :** L'identifiant unique du groupe dans le domaine (ex: -Identity "GG-EU-IT-Users")
- **DistinguishedName :** Le chemin complet dans l'AD (ex: -Identity "CN=GG-EU-IT-Users,OU=Groups,OU=EU,DC=maxtec,DC=be")
- **GUID :** L'identifiant unique global (ex: -Identity "123e4567-e89b-12d3-a456-426614174000")

```
# Maintenant on peut accéder facilement à ses propriétés (créez de valeurs pour les champs manquants dans AD, car le script est simplifié et crée des utilisateurs qui n'ont pas toutes les propriétés)
Write-Host $utilisateur.Surname
Write-Host $utilisateur.GivenName
Write-Host $utilisateur.Email
```

Note sur la langue : Même si votre interface AD est en français, les noms des propriétés dans PowerShell sont toujours en anglais. Utilisez donc **GivenName** (prénom), **Surname** (nom de famille), **Title** (titre), etc. Ces noms sont standardisés et ne changent pas avec la langue de l'interface.

Mais quelles sont les propriétés disponibles ?

Avant de pouvoir rechercher des informations spécifiques, il est important de connaître les attributs/propriétés disponibles. Ces propriétés sont aussi accessibles dans le menu **Propriétés > Editeur d'attributs** dans **Utilisateurs et groupes d'AD** (il faut activer l'option **Fonctionnalités avancées** dans le menu principal de **Utilisateurs et groupes d'AD**)

```
Get-ADUser -Filter * -Properties * | Get-Member -MemberType Property
```

Avec **Filter** on peut rechercher des éléments à partir des valeurs de leurs propriétés. Par **exemple, on peut rechercher des utilisateurs par leur nom, leur titre, leur service** ('department' en anglais. La propriété est dans l'onglet **Organisation** dans les propriétés des utilisateurs), etc.

```
# Exemple 1: Rechercher tous les utilisateurs dont le SamAccountName
commence par 'Ri' (observez que le script n'a pas crée les prénoms ni les
noms des utilisateurs)
Get-ADUser -Filter {SamAccountName -like 'Ri*'} -Properties GivenName |
Select-Object Name, SamAccountName, GivenName

# Exemple 2: Rechercher les utilisateurs créés après le 1er janvier 2023
$date = Get-Date "01/01/2023"
Get-ADUser -Filter {WhenCreated -ge $date} -Properties WhenCreated |
Select-Object Name, SamAccountName, WhenCreated
```

2. ♦ Les tableaux : vos collections d'objets

Un tableau est une collection d'éléments. Imaginez un classeur avec plusieurs tiroirs numérotés.

Création d'un tableau simple

```
# Créer un tableau de noms d'utilisateurs (attention à la notation de @ et
parenthèses)
$utilisateurs = @("jean.dupont", "marie.martin", "pierre.durand")

# Afficher le premier élément (les indices commencent à 0)
$utilisateurs[0]

# Afficher le dernier élément
$utilisateurs[-1]
```

Exercice 2.1 : Créez et manipulez un tableau

1. Créez un tableau contenant les noms de serveurs de votre réseau (par exemple: "dns1", "fileserver", "webserver")
2. Affichez le deuxième serveur du tableau
3. Ajoutez un quatrième serveur avec `$tableauServeurs += "mailserver"`
4. Faites une boucle `foreach` pour afficher chaque serveur sans utiliser le pipeline
5. Affichez chaque élément du tableau avec le pipeline

Tableaux d'objets AD

En PowerShell, les résultats de nombreuses commandes sont automatiquement des tableaux :

```
# Récupérer tous les utilisateurs du service Comptabilité: tableau d'objets
ADUser
$comptabilite = Get-ADUser -Filter *

# Combien d'utilisateurs avons-nous ? (nombre d'objets dans le tableau)
Write-Host $comptabilite.Count
```

```
# Accéder au premier utilisateur (accès au premier objet du tableau)
Write-Host $comptabilite[0].Name
```

Exercice 2.2 : Manipuler des collections d'objets AD

1. Récupérez tous les groupes dont le nom contient "GG-EU" (utiliser `Filter` - expliqué plus haut - et `like`)
2. Affichez le nombre de groupes trouvés
3. Affichez uniquement le nom du premier et du dernier groupe

3. ♦ Les boucles : répéter des actions

Les boucles permettent de répéter une action pour chaque élément d'une collection. C'est comme traiter un dossier de documents un par un.

La boucle ForEach

```
# Obtenir tous les utilisateurs
$Utilisateurs = Get-ADUser -Filter *

# Afficher chaque élément avec une boucle
foreach ($Utilisateur in $Utilisateurs) {
    Write-Host $Utilisateur
}

> Le bloc `{ }` contient les actions à répéter pour chaque élément.

# Voici une autre version qui utilise le pipeline. Le pipeline (`|`) permet
de chaîner des commandes.
# Le symbole $_ représente l'élément actuel dans la boucle.
$Utilisateurs | ForEach-Object { Write-Host $_ }
# $_ est juste un alias pour l'élément actuel dans la boucle. On peut en
plus accéder aux propriétés de l'élément, par exemple: $_.Name,
$_SamAccountName, $_.Title, etc.
$Utilisateurs | ForEach-Object { Write-Host $_.Name }
```

Exercice 3.1 : Votre première boucle

1. Récupérez tous les groupes de sécurité dont le nom commence par "GG-" (groupes globaux)
2. Créez une boucle qui affiche pour chacun : "Le groupe [Nom] a la description: [Description]"
3. Modifiez votre boucle pour n'afficher que les groupes créés après le 1er janvier 2023

Exercice 3.2 : Utiliser le pipeline

1. Utilisez le pipeline pour lister tous les groupes de sécurité
2. Pour chaque groupe, affichez son nom et le nombre de membres
3. Bonus : triez les résultats par nombre de membres (indice : `Sort-Object`)

4. ♦ Les conditions : prendre des décisions

Les conditions permettent d'exécuter du code uniquement si certains critères sont remplis. C'est comme suivre un arbre de décision.

Structure If-Else

Les conditions `if` permettent d'exécuter du code uniquement si une condition est vraie. Voici quelques exemples simples :

```
# Exemple 1: Condition simple
$estActif = $true
if ($estActif) {
    Write-Host "Le compte est actif"
}

# Exemple 2: Condition avec if/else
$nombreUtilisateurs = 5
if ($nombreUtilisateurs -gt 10) {
    Write-Host "Plus de 10 utilisateurs"
} else {
    Write-Host "10 utilisateurs ou moins"
}

# Exemple 3: Vérifier si un utilisateur existe
$nomUtilisateur = "jean.dupont"
if (Get-ADUser -Filter {SamAccountName -eq $nomUtilisateur} -ErrorAction SilentlyContinue) {
    Write-Host "L'utilisateur $nomUtilisateur existe"
} else {
    Write-Host "L'utilisateur $nomUtilisateur n'existe pas"
}

# Exemple 4: Vérifier si une valeur est vide
$description = ""
if ([string]::IsNullOrEmpty($description)) {
    Write-Host "La description est vide"
} else {
    Write-Host "La description contient: $description"
}
```

Les opérateurs de comparaison courants sont: `-eq` (égal), `-ne` (différent), `-gt` (supérieur), `-lt` (inférieur), `-ge` (supérieur ou égal), `-le` (inférieur ou égal).

Exercice 4.1 : Conditions simples

1. Récupérez un utilisateur de votre choix
2. Vérifiez s'il a une adresse email renseignée (modifiez les utilisateurs dans AD si besoin)
3. Affichez un message approprié selon le cas

Conditions multiples

```
# Vérifier le statut d'un utilisateur
$user = Get-ADUser -Identity "Richard" -Properties Enabled, LockedOut

if ($user.Enabled -eq $false) {
    Write-Host "Le compte est désactivé"
} elseif ($user.LockedOut -eq $true) {
    Write-Host "Le compte est verrouillé"
} else {
    Write-Host "Le compte est actif et utilisable"
}
```

Exercice 4.2 : Analyse d'utilisateurs

1. Créez un script qui demande un nom d'utilisateur et indique :
 - Si son compte est activé ou désactivé
 - S'il est membre du groupe "GG-EU-IT-Admins"
 - Si son mot de passe n'expire jamais

Exemple pratique : Rechercher un utilisateur

Voici un exemple de script qui combine variables, conditions et propriétés AD pour rechercher un utilisateur et afficher ses informations.

```
# Demander le nom d'utilisateur à rechercher (Read-Host attendra la saisie
de l'utilisateur)
$nomUtilisateurRecherche = Read-Host "Entrez le nom d'utilisateur à
rechercher"

# Récupérer l'utilisateur dans une variable (filtrer par SamAccountName) et
obtenir les propriétés GivenName, Surname et Title
$utilisateurTrouve = Get-ADUser -Filter {SamAccountName -eq
$nomUtilisateurRecherche} -Properties GivenName, Surname, Title,
Department, WhenCreated

# Vérifier si l'utilisateur existe
if ($utilisateurTrouve) {
    Write-Host "Utilisateur trouvé !"
    Write-Host "Prénom : $($utilisateurTrouve.GivenName)"
    Write-Host "Nom : $($utilisateurTrouve.Surname)"
    Write-Host "Titre : $($utilisateurTrouve.Title)"
    Write-Host "Service : $($utilisateurTrouve.Department)"
    Write-Host "Date de création : $($utilisateurTrouve.WhenCreated)"

    # Vérifier si l'utilisateur fait partie du service IT
    if ($utilisateurTrouve.Department -eq "IT") {
        Write-Host "Cet utilisateur fait partie du service informatique" -
ForegroundColor Green
    }
}
```

```

    }
} else {
    Write-Host "Aucun utilisateur trouvé avec ce nom." -ForegroundColor Red
}

```

Important : Modifier les valeurs dans une variable n'a aucun effet sur l'objet réel dans Active Directory. Par exemple, si vous faites `$utilisateurTrouve.Title = "Nouveau Titre"`, cela change uniquement la valeur dans votre variable locale, pas dans AD. Pour modifier réellement un objet AD, vous devez utiliser des commandes spécifiques comme `Set-ADUser` que nous verrons plus tard.

Exercice 4.3 : Améliorer le script de recherche

1. Modifiez le script ci-dessus pour afficher également si le compte est activé ou désactivé
2. Ajoutez une condition pour afficher un message spécial si l'utilisateur fait partie du service "Comptabilité"
3. Affichez les trois premiers groupes dont l'utilisateur est membre

5. ♦ Mini-projet : Rapport d'audit AD

Objectif

Créer un rapport qui liste **tous les utilisateurs** d'une **Unité d'Organisation (OU)** spécifique, avec leur statut de compte et leurs groupes principaux.

```

# Demander le nom de l'OU à l'utilisateur
$nomOU = Read-Host "Entrez le nom de l'OU à auditer (ex: Ventes, RH...)"

# Construire le chemin de l'OU (adapté au domaine computerelectronics.be)
# Note: Le chemin exact dépend de votre structure AD
$cheminOU = "OU=Users,OU=$nomOU,OU=EU,DC=computerelectronics,DC=be"

# Récupérer les utilisateurs de cette OU
$users = Get-ADUser -Filter * -SearchBase $cheminOU -Properties Enabled,
MemberOf

# Vérifier si des utilisateurs ont été trouvés
if ($users.Count -eq 0) {
    Write-Host "Aucun utilisateur trouvé dans l'OU $nomOU" -ForegroundColor
Yellow
} else {
    # Afficher un en-tête
    Write-Host "=== RAPPORT D'AUDIT : OU $nomOU ===" -ForegroundColor Cyan
    Write-Host "Nombre d'utilisateurs : $($users.Count)" -ForegroundColor
Cyan
    Write-Host "-----"

    # Pour chaque utilisateur
    foreach ($user in $users) {
        # Statut du compte

```

```
$statut = if ($user.Enabled) {"ACTIF"} else {"DÉSACTIVÉ"}
$couleur = if ($user.Enabled) {"Green"} else {"Red"}

# Afficher les informations de base
Write-Host "UTILISATEUR : $($user.Name) - $statut" -ForegroundColor
$couleur

# Récupérer et afficher les 3 premiers groupes
$groupes = Get-ADPrincipalGroupMembership -Identity
$user.SamAccountName | Select-Object -First 3
Write-Host "  Groupes principaux :"
foreach ($groupe in $groupes) {
    Write-Host "    - $($groupe.Name)"
}

Write-Host "-----"
}
```

Exercice final : Personnalisez le rapport

1. Adaptez le script pour qu'il affiche également :
 - La date de dernière connexion (LastLogonDate)
 - Si le mot de passe expire ou non
2. Ajoutez une condition pour mettre en évidence les comptes inactifs depuis plus de 30 jours
3. Bonus : Permettez de filtrer les utilisateurs par leur statut (actif/inactif)